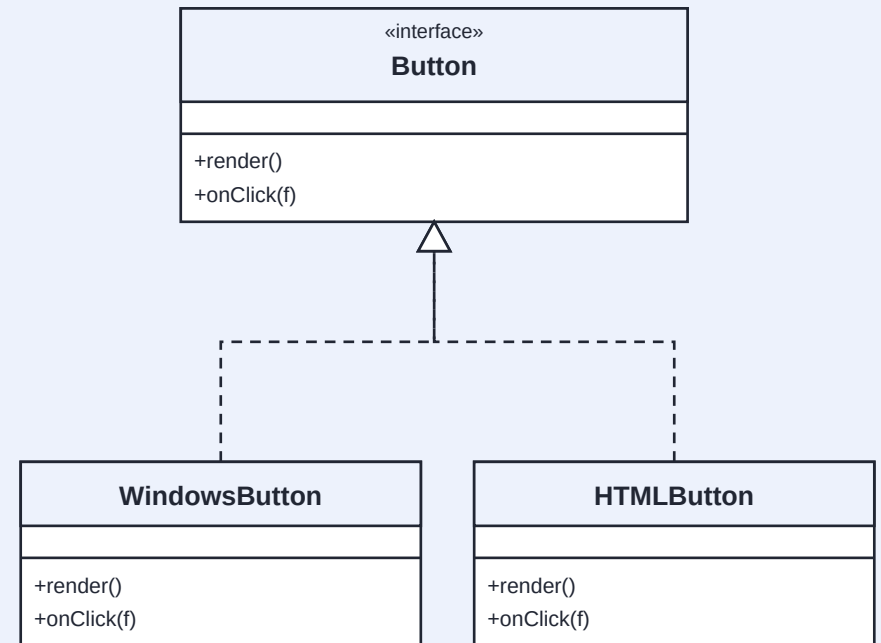


UML Class Diagram Rule Book

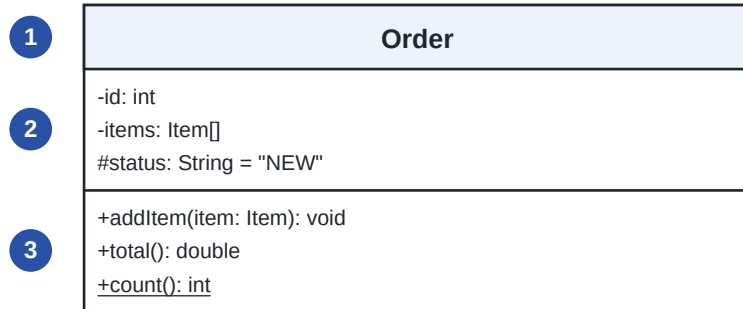
Notation, relationships and drawing rules, with examples from the creational design patterns.

CONTENTS

1 Anatomy of a class	2
2 Members, visibility and markers	3
3 Relationships	4
4 Multiplicity and roles	5
5 Notes and stereotypes	6
6 Layout and style rules	7
7 Common mistakes	8
8 Quick reference	9

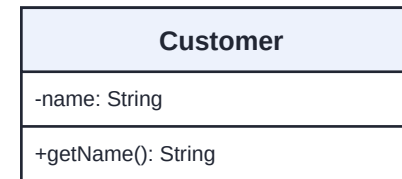


Anatomy of a class

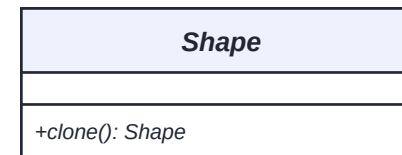


1. Name compartment: the class name, bold and centered, written in PascalCase.
2. Attribute compartment: one line per field, written as visibility name: type = default.
3. Method compartment: one line per operation, written as visibility name(parameter: type): returnType.
4. An underlined member is static. Here count() belongs to the class, not to an object.

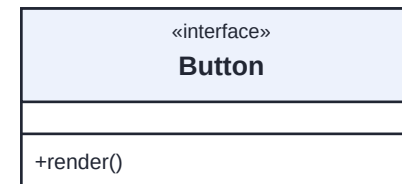
CLASS KINDS



Concrete class. Bold upright name. Can be instantiated.



Abstract class. Name and abstract methods in italics. Cannot be instantiated; subclasses complete it.



Interface. The «interface» stereotype sits above the name. Only method signatures, no fields or bodies.

Members, visibility and markers

VISIBILITY SYMBOLS

Symbol	Name	Example
+	Public	+getName(): String
-	Private	-balance: double
#	Protected	#id: int
~	Package	~helper(): void

STYLE MARKERS

Marker	Meaning	Example
<i>Italic</i>	Abstract class or abstract method (no body).	+createProduct(): <i>Product</i>
<u>Underline</u>	Static member. It belongs to the class itself.	+getInstance(): Singleton
«interface»	The type is an interface.	«interface» Button

MEMBER SYNTAX

Attribute

visibility name: type [multiplicity] = default

-balance: double = 0

-items: Item[0..*]

Method

visibility name(param: type, ...): returnType

+deposit(amount: double): void

+Order(id: int)

- Name first, then a colon, then the type.
- Constructors repeat the class name and have no return type.
- Leave out parameters and trivial getters when they only add noise.

Relationships

Symbol	Name	Meaning	Typical code
	Association	Plain solid line. Objects of the two classes are linked. Read: A knows B.	<code>private Engine engine;</code>
	Directed association	Solid line, open arrow. A holds a reference to B; B knows nothing about A.	<code>private Prototype prototype;</code>
	Aggregation	Hollow diamond at the whole. Has-a; the part can live without the whole.	<code>Director(Builder builder)</code>
	Composition	Filled diamond at the whole. Owns-a; the part is created and destroyed with the whole.	<code>this.room = new Room();</code>
	Inheritance	Solid line, hollow triangle at the parent. Is-a between two classes.	<code>class Circle extends Shape</code>
	Realization	Dashed line, hollow triangle at the interface. The class implements it.	<code>class WinButton implements Button</code>
	Dependency	Dashed line, open arrow at the used class. Temporary use: parameter, local variable or creation.	<code>return new Product();</code>

Multiplicity and roles

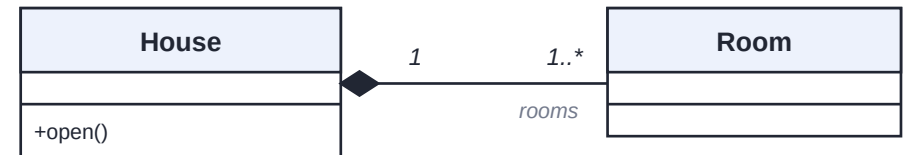
MULTIPLICITY VALUES

Notation	Meaning
1	Exactly one
0..1	Zero or one (optional)
* or 0..*	Zero or more
1..*	One or more
2..4	A range, here two to four

- Write the number at the end of the line next to the class it describes.
- Read it as: how many objects of that class relate to ONE object at the other end.
- A role name (for example orders) goes next to the same end and names what that class plays.
- An association name (for example places) goes in the middle of the line, with a verb.



One Customer places zero or more Orders. Each Order belongs to exactly one Customer.



A House owns one or more Rooms. Rooms are destroyed together with their House.

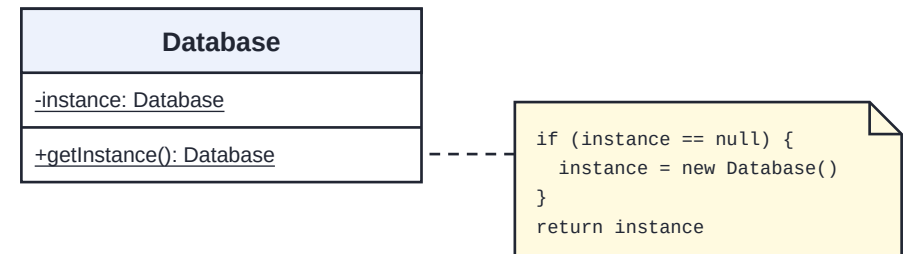
Notes and stereotypes

- A note is a rectangle with a folded corner. Attach it with a dashed line that has no arrowhead. Use it for pseudocode, constraints and remarks that do not fit in a member list.
- A stereotype is a keyword in guillemets, written «Like This». It classifies an element or labels a relationship.
- Put a relationship stereotype on or next to its line, in small italic text.

COMMON STEREOTYPES

Stereotype	Meaning
«interface»	A type with only method signatures.
«enumeration»	A fixed set of named constants.
«creates»	Dependency where the source instantiates the target.
«uses»	Dependency where the source calls the target.

EXAMPLE: NOTE



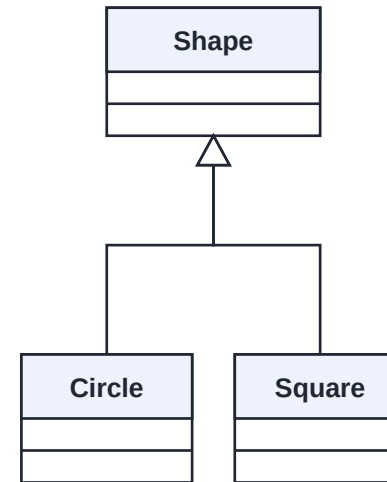
EXAMPLE: STEREOTYPED DEPENDENCY



Layout and style rules

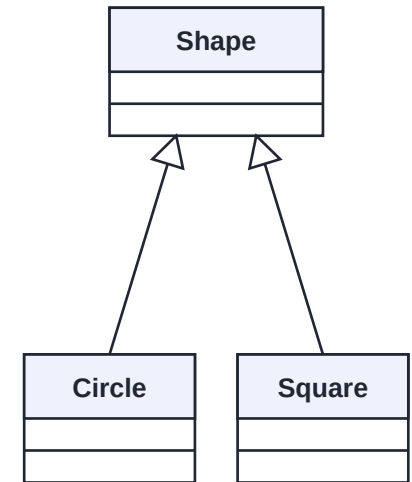
1. Place parents and interfaces above, children and implementations below. Inheritance arrows point up.
2. Use horizontal and vertical lines. Avoid diagonals.
3. Avoid crossings. If one is unavoidable, cross at a right angle.
4. Join several children to one parent with a single shared triangle.
5. Align classes of the same level on one row and give siblings the same width.
6. Group what belongs together, for example the factory family next to the product family.
7. Solid lines show structure. Dashed lines show weaker or implementing links.
8. Name classes with nouns in PascalCase, members in camelCase, and label relationships with verbs or roles.
9. Show only what the diagram needs. Hide trivial getters and private helpers in overview diagrams.
10. One diagram, one purpose. Split a diagram that needs a legend to be understood.

DO



One shared triangle, straight lines.

DON'T



Separate triangles, diagonal lines.

Common mistakes

Mistake	Why it is wrong	Fix
Solid line for implementing an interface	A solid triangle line means class inheritance.	Use a dashed line with a hollow triangle (realization).
Diamond on the part	The diamond always marks the whole, the owner.	Move the diamond to the end touching the whole.
Hollow vs filled diamond mixed up	Hollow means aggregation, filled means composition (ownership).	Filled only when the part cannot outlive the whole.
Arrow pointing the wrong way	The head points at the class being used, extended or implemented.	Draw from the dependent class toward the one it needs.
Abstract members in upright text	Readers cannot tell which methods need implementing.	Write abstract classes and methods in italics.
Association for every link	Temporary use is not a stored relationship.	Use a dashed dependency for parameters, locals and creation.
Overloaded diagram	Too many classes hide the idea being shown.	Split by purpose and show only relevant members.

Quick reference

RELATIONSHIP LEGEND

	Association
	Directed association
	Aggregation
	Composition
	Inheritance
	Realization
	Dependency

Visibility: + public - private # protected ~ package

Italic = abstract Underline = static

BEFORE YOU SHARE A DIAGRAM

- Abstract names and members are italic.
- Every diamond sits on the whole.
- Arrowheads point at the used or parent class.
- Multiplicities are on the correct ends.
- Lines are straight with few crossings.
- Every class earns its place.

WHERE EACH RELATIONSHIP APPEARS IN THE CREATIONAL PATTERNS

Pattern	Relationships used
Factory Method	Inheritance, realization, dependency «creates»
Abstract Factory	Realization, directed association, dependencies
Builder	Aggregation, realization, dependency «creates»
Prototype	Directed association, realization, inheritance
Singleton	Self-association, dependency, note